

Milestone 2 Report

Smart MCQ Solver Challenge - Tokenization, Embeddings & Generative Models

This report covers the approach and reasoning behind each question in Milestone 2. The notebook explores four distinct toolsets - HuggingFace datasets, BERT tokenization and embeddings, sentence transformers, and generative/zero-shot models - building up from basic data loading to running a small generative language model on the MCQ task.

Section 1 — Data Loading & Tokenization (Q1–Q5)

Q1. Load with HuggingFace datasets + combined_text column

Used `load_dataset('csv', ...)` instead of `pd.read_csv` as required, then `.map()` to concatenate prompt and A with a single space. Character length checked with Python's built-in `len()` on the string at zero-index 51.

↳ The `.map()` approach applies the lambda to every row in one pass without writing a loop — same idea as `pandas.apply()` but it's the idiomatic HuggingFace datasets pattern.

Q2. BERT tokenizer vocabulary size

Loaded `bert-base-uncased` with `AutoTokenizer.from_pretrained()`. The `vocab_size` property directly returns the number of unique subword tokens hardcoded into the tokenizer — no computation needed.

↳ `bert-base-uncased` uses `WordPiece` tokenization. The vocabulary is fixed at training time and includes whole words, subword pieces (e.g. '##ing'), and special tokens like `[CLS]`, `[SEP]`, `[PAD]`.

Q3. [SEP] token integer ID

Accessed `tokenizer.sep_token_id` directly — a built-in property of every HuggingFace tokenizer that returns the integer ID for the separator token without needing to look it up in the vocab manually.

↳ `[SEP]` marks the boundary between sentences in BERT's input format. For single-sentence tasks it appears at the end; for sentence-pair tasks it separates the two sequences.

Q4. Shape of input_ids tensor — full prompt column

Passed the entire prompt column list to `tokenizer()` with `padding='max_length'`, `truncation=True`, `max_length=128`, `return_tensors='pt'`. The shape is `(number_of_training_rows, 128)` — every row padded or truncated to exactly 128 tokens.

↳ Padding to `max_length` ensures every row becomes the same length so they can be stacked into a rectangular tensor. Truncation handles any prompts longer than 128 tokens by cutting them off.

Q5. Dimensionality per attention head

Pure arithmetic: BERT's hidden size (768) divided by the number of attention heads (12) = 64 dimensions per head. No code needed beyond the division.

↳ Each attention head operates in its own 64-dim subspace. The heads run in parallel and their outputs are concatenated back to 768 before the next layer — this is the Multi-Head Attention design.

Section 2 — BERT Embeddings (Q6–Q8)

Q6. Shape of last_hidden_state for row 0 prompt

Loaded `AutoModel.from_pretrained('bert-base-uncased')`, tokenized the row 0 prompt with default settings (no padding or truncation specified), then passed through the model with `torch.no_grad()` to avoid building a gradient graph. The shape is `(1, sequence_length, 768)` — batch of 1, however many tokens the prompt tokenizes to, 768 hidden dims.

↳ The sequence length varies here because we're not padding — it's exactly the number of tokens in that specific prompt including `[CLS]` and `[SEP]`. Used `model.eval()` to disable dropout so outputs are deterministic.

Q7. Sum of first 5 CLS embedding values

The [CLS] token is always at position 0 in BERT's output sequence. Extracted `last_hidden_state[0, 0, :]` (batch 0, token index 0, all 768 dims), then sliced `[:5]` and summed. Rounded to 4 decimal places with Python's `round()`.

↳ The [CLS] token is designed to aggregate sequence-level information during pre-training. In classification fine-tuning it's used as the sentence representation — though for this question we're just inspecting its raw pre-trained values.

Q8. Attention weight CLS→fusion in last layer, head 0

Loaded the model with `output_attentions=True`, tokenized the exact string 'Light-ion fusion is a technique.', and passed through. Found the token index for 'fusion' by converting `input_ids` to tokens and calling `.index('fusion')`. Then accessed `outputs.attentions[-1][0, 0]` — last layer, batch 0, head 0 — and read the attention weight at `[0, fusion_idx]` (row = CLS, col = fusion). Rounded to 4 decimal places.

↳ `outputs.attentions` is a tuple of length 12 (one per layer), each shaped (batch, heads, seq, seq). The value at `[cls_idx, token_idx]` is the fraction of attention that [CLS] allocates to that token — all rows sum to 1.0 due to the softmax inside attention.

Section 3 — Sentence Transformers (Q9–Q10)

Q9. Cosine similarity: prompt vs option B at row 0 using SentenceTransformer

Loaded `sentence-transformers/all-MiniLM-L6-v2`, encoded the row 0 prompt and row 0 option B separately with `.encode()`, then computed similarity with `st_util.cos_sim()` as specified (not manual dot-product). Rounded to 4 decimal places.

↳ `cos_sim()` from `sentence_transformers` handles the tensor shape automatically — it returns a (1,1) tensor so we call `.item()` to get the scalar. Using the library's own `cos_sim` rather than `sklearn` or `numpy` ensures the result matches exactly what the task expects.

Section 4 — Zero-Shot Classification & Generative AI (Q11–Q13)

Q11. Zero-shot classification — Softmax (single-label)

Initialized pipeline('zero-shot-classification') which defaults to `facebook/bart-large-mnli`. Passed the row 1 prompt with [A, B, C] as `candidate_labels`. The top-ranked option's probability score is the answer — this is a softmax over the 3 candidates so the 3 scores sum to ~1.0.

↳ BART-MNLI frames each candidate as a hypothesis: 'This example is [candidate]' and scores the entailment probability. With `multi_label=False` (default), a single softmax normalizes across all candidates so they compete.

Q12. Zero-shot classification — Sigmoid (multi-label) + absolute difference

Same call with `multi_label=True` added. This switches from a shared softmax to independent sigmoid per candidate, so each score is computed in isolation and the 3 scores no longer sum to 1.0. The answer is `|sum(softmax_scores) - sum(sigmoid_scores)|`.

↳ The difference captures how much the two probability formulations disagree. Softmax forces competition (one label 'wins'), sigmoid allows each label to be independently likely — which matters when multiple options could plausibly be correct, as in some MCQ questions.

Q13. Generative AI — flan-t5-small text2text output for row 0

Tried to load pipeline('text2text-generation', model='google/flan-t5-small') but found text2text-generation is not available

Overall, Milestone 2 moves from hand-crafted similarity features (TF-IDF, content-word length) to pretrained representations (BERT token embeddings, sentence-level MiniLM embeddings) and then to models that can directly reason over the question text (zero-shot BART-MNLI, generative flan-T5).